

Responsive web design :

Responsive web design (RWD) is a web design approach to make web pages render well on all screen sizes and resolutions while ensuring good usability. It is the way to design for a multi-device web.

It uses HTML and CSS to adjust, hide, shrink, or enlarge web elements automatically, ensuring that a website looks great on all devices — desktops, tablets, and mobile phones.

Principles of Responsive Design :

Principles of web design are the basic ideas that make a website look good, work well, and provide a great user experience.

Principle	Description	Example
1. Fluid Grid Layout	Uses relative units (<code>%</code> , <code>em</code> , <code>rem</code> , <code>vw</code>) instead of fixed (<code>px</code>) to make layout flexible.	<code>width: 50%;</code> instead of <code>width: 500px;</code>
2. Flexible Images & Media	Images scale according to screen size using CSS properties like <code>max-width: 100%</code> .	<code>img { max-width: 100%; height: auto; }</code>
3. Media Queries	Special CSS rules that apply styles based on device characteristics like width or orientation.	<code>@media (max-width: 768px) { ... }</code>
4. Mobile-First Design	Start designing for small screens, then add more features for larger screens.	Use <code>min-width</code> breakpoints in CSS
5. Responsive Typography	Font sizes adjust automatically for readability.	<code>font-size: clamp(1rem, 2vw, 2rem);</code>

Fluid Grid Layout

A **fluid grid** uses percentage-based widths instead of fixed pixels, so elements resize automatically with the screen.

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fluid Grid Example</title>
  <style>
    * {
      box-sizing: border-box;
    }

    .container {
      display: flex;
      gap: 10px;
      padding: 10px;
    }

    .box {
      background-color: steelblue;
      color: white;
      text-align: center;
      padding: 20px;
      width: 33.33%;
      /* fluid width */
    }

  </style>
</head>

<body>

  <h2>Fluid Grid Layout</h2>
  <div class="container">
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
    <div class="box">Box 3</div>
  </div>

</body>

</html>
```

Flexible Images:

Images automatically resize with their container while maintaining aspect ratio.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Flexible Images</title>
<style>
  .wrapper {
    width: 80%;
    margin: auto;
  }

  img {
    max-width: 100%;
    height: auto;
    border-radius: 10px;
  }
</style>
</head>
<body>

<h2>Flexible Images</h2>
<div class="wrapper">
  
</div>

</body>
</html>
```

Media Queries:

Media queries are CSS techniques that allow developers to apply different sets of styles based on the characteristics of the device being used to view the content. They are the most critical tool for creating breakpoints.

Breakpoint:

A breakpoint is a condition inside a media query where your CSS starts behaving differently based on screen size.

Two Types of Breakpoints

1. Max-width Breakpoints

Trigger styles when the screen is *smaller* than a certain width.

Syntax :

```
@media (max-width: 768px) { ... }
```

2. Min-width Breakpoints

Trigger styles when the screen is *larger* than a certain width.

Syntax :

```
@media (min-width: 768px) { ... }
```

Device Type	Typical Screen Width	Example Media Query
Extra Small (Mobile)	max-width: 480px	@media (max-width: 480px)
Small (Large Mobile / Small Tablet)	max-width: 600px	@media (max-width: 600px)
Medium (Tablet)	max-width: 768px	@media (max-width: 768px)
Large (Laptop)	max-width: 1024px	@media (max-width: 1024px)

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Media Query Example</title>
<link rel="stylesheet" href="./media-style1.css">
</head>
<body>
```

```
<h1>Responsive Design</h1>
<p>Resize the browser window to see color change.</p>
</body>
</html>
```

media-style1.css

```
/* Default style - Desktop */
body {
  background-color: lightyellow;
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 50px;
}

/* Tablet breakpoint */
@media (max-width: 768px) {
  body {
    background-color: lightgreen;
  }
}

/* Mobile breakpoint */
@media (max-width: 480px) {
  body {
    background-color: lightblue;
  }
}
```

Mobile First Design:

Mobile-first design is a web development approach that starts by creating websites for **small screens (mobiles)** first, then gradually enhances them for **larger screens** like tablets and desktops. This ensures a **good user experience** on all devices. It's a form of **progressive enhancement**, focusing on core features first and adding more as screen size increases.

Advantages:

1. Ensures a better experience for mobile users (most web traffic is mobile).
2. Makes websites faster and more focused.
3. Easier to scale up design than to shrink down.
4. Google uses **mobile-first indexing**, so mobile-optimized sites rank better in search results.
5. Mobile-first design focuses on lightweight content and fewer elements, improving performance.

Responsive Typography:

Responsive typography refers to text whose **font size and layout automatically adapt** to various screen sizes, ensuring that the content remains **clear and easy to read** on **mobile phones, tablets, and desktop devices**.

Three ways to create responsive Typography

1. Use Relative Units (**em**, **rem**, **%**)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Typography - Relative Units</title>
<link rel="stylesheet" href="./responsive-typography.css">
</head>
<body>
  <header>
    <h1>Responsive Typography (rem & em)</h1>
  </header>
  <main>
    <p>This text uses relative units like rem and em to scale naturally with the device size or user zoom settings.</p>
  </main>
</body>
```

```
</html>
```

responsive-typography.css

```
/* Base font size */
html {
  font-size: 16px;
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 20px;
}

/* Headings and paragraph use relative units */
h1 {
  font-size: 2rem; /* 2 × 16px = 32px */
}

p {
  font-size: 1rem; /* 1 × 16px = 16px */
  max-width: 600px;
  margin: auto;
}
```

2. Use Media Queries

Change font sizes for different screen widths.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Typography - Media Queries</title>
<link rel="stylesheet" href="./responsive-typography2.css">
```

```
</head>
<body>
  <header>
    <h1>Responsive Typography with Media Queries</h1>
  </header>
  <main>
    <p>Font sizes change at different screen widths using CSS media queries.</p>
  </main>
</body>
</html>
```

responsive-typography2.css

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  margin: 0;
  padding: 20px;
  font-size: 16px; /* Default mobile font */
}

/* Tablet view */
@media (min-width: 768px) {
  body {
    font-size: 18px;
  }
}

/* Desktop view */
@media (min-width: 1024px) {
  body {
    font-size: 20px;
  }
}

h1 {
  font-size: 1.8em;
}

p {
  max-width: 700px;
  margin: auto;
}
```



```
}
```

3. Use `clamp()` Function (Modern CSS)

The `clamp()` function automatically scales font size between a minimum, preferred, and maximum value.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Typography - clamp()</title>
<link rel="stylesheet" href="./responsive-typography3.css">
</head>
<body>
  <header>
    <h1>Responsive Typography with clamp()</h1>
  </header>
  <main>
    <p>Resize the browser window to see the font size automatically scale
between a minimum and maximum range.</p>
  </main>
</body>
</html>
```

responsive-typoography3.css

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 20px;
}

/* Font size scales automatically based on viewport width */
h1 {
  font-size: clamp(1.5rem, 5vw, 3rem);
}
```

```
p {  
  font-size: clamp(1rem, 3vw, 1.5rem);  
  max-width: 700px;  
  margin: auto;  
}
```

Media-query

A **media query** in CSS is a rule that allows you to apply **different styles** to your webpage depending on the **device's characteristics**, such as:

- Screen **width** or **height**
- Device **orientation** (portrait or landscape)
- Screen **resolution**
- Display **type** (screen, print, etc.)
-

Component of Media Query

1. **@media** – Keyword used to define the query.
2. **Media Type** – Defines where the style applies.
 - Common types:
 - **screen** → for monitors, tablets, phones
 - **print** → for printers
 - **speech** → for screen readers
3. **Condition (Expression)** – A test based on device features, written in parentheses **()**.

- Example: `(max-width: 768px)`

4. **CSS Rules** – The styles that apply when the condition is true.

Media Features:

Property	Description	Example
<code>width</code>	Width of the viewport	<code>(max-width: 600px)</code>
<code>height</code>	Height of the viewport	<code>(max-height: 800px)</code>
<code>orientation</code>	Portrait or landscape	<code>(orientation: portrait)</code>
<code>resolution</code>	Screen resolution (DPI)	<code>(min-resolution: 300dpi)</code>
<code>aspect-ratio</code>	Ratio of width to height	<code>(max-aspect-ratio: 16/9)</code>

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Card Layout</title>
  <link rel="stylesheet" href="card-layout.css">
</head>
<body>
  <header>
    <h1>Popular Products</h1>
  </header>

  <main class="card-container">
    <section class="card">
      
      <h2>Laptop</h2>
      <p>High-quality Display</p>
      <button>Add to Cart</button>
    </section>

    <section class="card">
      
      <h2>Book</h2>
      <p>HTML CSS</p>
      <button>Add to Cart</button>
    </section>

    <section class="card">
      
      <h2>Coffee MUG</h2>
      <p>Stylish Coffe Mug for derverloper.</p>
      <button>Add to Cart</button>
    </section>

    <section class="card">
      
```

```
    <h2>Cars</h2>
    <p>High performance with vintage cards.</p>
    <button>Add to Cart</button>
  </section>
</main>

<footer>
  <p>&copy; 2025 MyStore - All rights reserved.</p>
</footer>
</body>
</html>
```

```
/* =====
   Base Styling
===== */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background-color: #f6f6f6;
}

header {
  background-color: #232f3e;
  color: white;
  text-align: center;
  padding: 20px;
}

footer {
  background-color: #232f3e;
  color: white;
  text-align: center;
  padding: 15px;
  font-size: 14px;
}

h1 {
  margin: 0;
```

```

}

/* =====
   Card Container Styling
===== */
.card-container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 20px;
  padding: 30px;
}

.card {
  background-color: white;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  text-align: center;
  padding: 20px;
  transition: transform 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
}

.card img {
  width: 100%;
  height: 160px;
  border-radius: 8px;
  object-fit: cover;
}

.card h2 {
  font-size: 18px;
  color: #333;
  margin: 10px 0;
}

.card p {
  font-size: 14px;
  color: #555;
  margin-bottom: 15px;
}

.card button {

```

```
background-color: #ff9900;
color: white;
border: none;
padding: 10px 20px;
border-radius: 6px;
cursor: pointer;
font-weight: bold;
}

.card button:hover {
  background-color: #e68a00;
}

/* Media Queries */

/* Tablet View */
@media (max-width: 1024px) {
  .card-container {
    grid-template-columns: repeat(2, 1fr);
  }

  header h1 {
    font-size: 24px;
  }
}

/* Mobile View */
@media (max-width: 600px) {
  .card-container {
    grid-template-columns: 1fr;
    padding: 15px;
  }

  .card {
    padding: 15px;
  }

  header h1 {
    font-size: 20px;
  }

  .card h2 {
    font-size: 16px;
  }
}
```

Example2:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Responsive Website</title>
  <link rel="stylesheet" href="web-layout.css">
</head>
<body>

  <!-- Header -->
  <header>
    <h1>My Responsive Website</h1>
    <nav>
      <ul class="nav-links">
        <li><a href="#blog">Blog</a></li>
        <li><a href="#cards">Cards</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <!-- Hero Section -->
  <section class="hero">
    <h2>Welcome to My Website</h2>
    <p>Explore blogs, cards, and responsive design examples below!</p>
    <button class="hero-btn">Learn More</button>
  </section>

  <!-- Blog Section -->
  <section id="blog" class="blog-section">
    <h2>Latest Blog Posts</h2>
    <article class="blog-post">
      
      <h3>Understanding Responsive Design</h3>
      <p>Responsive web design ensures your website looks great on all
devices -- from phones to desktops. Learn how media queries and flexible
layouts make that possible.</p>
    </article>
```



```

    <article class="blog-post">
      
      <h3>Why Flexbox is Awesome</h3>
      <p>Flexbox makes aligning and distributing space among items easy and predictable in different screen sizes. It's one of the most powerful CSS layout tools.</p>
    </article>
  </section>

  <!-- Card Section -->
  <section id="cards" class="card-section">
    <h2>Featured Cards</h2>
    <div class="card-container">
      <div class="card">
        
        <h3>Card Title 1</h3>
        <p>This is a short description of card one. It's designed using pure CSS.</p>
        <button>Read More</button>
      </div>
      <div class="card">
        
        <h3>Card Title 2</h3>
        <p>This is a short description of card two. Hover to see animation effects.</p>
        <button>Read More</button>
      </div>
      <div class="card">
        
        <h3>Card Title 3</h3>
        <p>This is a short description of card three. Fully responsive layout.</p>
        <button>Read More</button>
      </div>
    </div>
  </section>

  <!-- Contact Form -->
  <section id="contact" class="contact-section">
    <h2>Contact Us</h2>
    <form>
      <label for="name">Full Name</label>
      <input type="text" id="name" placeholder="Enter your name"

```

```

required>

    <label for="email">Email</label>
    <input type="email" id="email" placeholder="Enter your email"
required>

    <label for="message">Message</label>
    <textarea id="message" placeholder="Write your message..." rows="5"
required></textarea>

    <button type="submit" class="submit-btn">Send Message</button>
  </form>
</section>

<!-- Footer -->
<footer>
  <p>(c) 2025 My Responsive Website | Designed by BridgeLabz</p>
</footer>

</body>
</html>

```

web-layout.css

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Responsive Website</title>
  <link rel="stylesheet" href="web-layout.css">
</head>
<body>

  <!-- Header -->
  <header>
    <h1>My Responsive Website</h1>
    <nav>
      <ul class="nav-links">
        <li><a href="#blog">Blog</a></li>
        <li><a href="#cards">Cards</a></li>
        <li><a href="#contact">Contact</a></li>

```

```

        </ul>
    </nav>
</header>

<!-- Hero Section -->
<section class="hero">
    <h2>Welcome to My Website</h2>
    <p>Explore blogs, cards, and responsive design examples below!</p>
    <button class="hero-btn">Learn More</button>
</section>

<!-- Blog Section -->
<section id="blog" class="blog-section">
    <h2>Latest Blog Posts</h2>
    <article class="blog-post">
        

        <h3>Understanding Responsive Design</h3>
        <p>Responsive web design ensures your website looks great on all
devices -- from phones to desktops. Learn how media queries and flexible
layouts make that possible.</p>
    </article>

    <article class="blog-post">
        

        <h3>Why Flexbox is Awesome</h3>
        <p>Flexbox makes aligning and distributing space among items easy
and predictable in different screen sizes. It's one of the most powerful CSS
layout tools.</p>
    </article>
</section>

<!-- Card Section -->
<section id="cards" class="card-section">
    <h2>Featured Cards</h2>
    <div class="card-container">
        <div class="card">
            
            <h3>Card Title 1</h3>
            <p>This is a short description of card one. It's designed using
pure CSS.</p>
            <button>Read More</button>
        </div>
        <div class="card">

```

```

        
        <h3>Card Title 2</h3>
        <p>This is a short description of card two. Hover to see
animation effects.</p>
        <button>Read More</button>
    </div>
    <div class="card">
        
        <h3>Card Title 3</h3>
        <p>This is a short description of card three. Fully responsive
layout.</p>
        <button>Read More</button>
    </div>
</div>
</section>

<!-- Contact Form -->
<section id="contact" class="contact-section">
    <h2>Contact Us</h2>
    <form>
        <label for="name">Full Name</label>
        <input type="text" id="name" placeholder="Enter your name"
required>

        <label for="email">Email</label>
        <input type="email" id="email" placeholder="Enter your email"
required>

        <label for="message">Message</label>
        <textarea id="message" placeholder="Write your message..." rows="5"
required></textarea>

        <button type="submit" class="submit-btn">Send Message</button>
    </form>
</section>

<!-- Footer -->
<footer>
    <p>(c) 2025 My Responsive Website | Designed by BridgeLabz</p>
</footer>

</body>
</html>

```

CheetSheet

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive CSS Cheat Sheet</title>
  <link rel="stylesheet" href="./cheetsheet2.css">
</head>
<body>
  <header>
    <h1>Responsive CSS Cheat Sheet</h1>
    <p>Quick reference for filters, media queries, responsive web design, and
typography.</p>
  </header>

  <main>
    <section>
      <h2>1. CSS Filters</h2>
      <table>
        <thead>
          <tr><th>Property</th><th>Description</th><th>Example</th></tr>
        </thead>
        <tbody>
          <tr><td data-label="Property">blur()</td><td
data-label="Description">Applies a blur effect.</td><td
data-label="Example">filter: blur(5px);</td></tr>
          <tr><td data-label="Property">brightness()</td><td
data-label="Description">Adjusts brightness.</td><td
data-label="Example">filter: brightness(1.2);</td></tr>
          <tr><td data-label="Property">contrast()</td><td
data-label="Description">Adjusts contrast level.</td><td
data-label="Example">filter: contrast(200%);</td></tr>
          <tr><td data-label="Property">grayscale()</td><td
data-label="Description">Converts to grayscale.</td><td
data-label="Example">filter: grayscale(100%);</td></tr>
          <tr><td data-label="Property">hue-rotate()</td><td
data-label="Description">Rotates hue color.</td><td
data-label="Example">filter: hue-rotate(90deg);</td></tr>
          <tr><td data-label="Property">invert()</td><td
data-label="Description">Inverts colors.</td><td data-label="Example">filter:
invert(100%);</td></tr>
          <tr><td data-label="Property">opacity()</td><td
```

```

data-label="Description">Adjusts transparency.</td><td
data-label="Example">filter: opacity(0.5);</td></tr>
  <tr><td data-label="Property">saturate()</td><td
data-label="Description">Increases color intensity.</td><td
data-label="Example">filter: saturate(2);</td></tr>
  <tr><td data-label="Property">sepia()</td><td
data-label="Description">Applies sepia tone.</td><td
data-label="Example">filter: sepia(100%);</td></tr>
  <tr><td data-label="Property">drop-shadow()</td><td
data-label="Description">Adds a shadow effect.</td><td
data-label="Example">filter: drop-shadow(5px 5px 10px gray);</td></tr>
</tbody>
</table>
</section>

<section>
  <h2>2. Responsive Web Design</h2>
  <table>
    <thead>
      <tr><th>Technique</th><th>Description</th><th>Example</th></tr>
    </thead>
    <tbody>
      <tr><td data-label="Technique">Fluid Layout</td><td
data-label="Description">Uses percentage-based widths.</td><td
data-label="Example">width: 80%;</td></tr>
      <tr><td data-label="Technique">Flexible Images</td><td
data-label="Description">Images resize within containers.</td><td
data-label="Example">img { max-width: 100%; height: auto; }</td></tr>
      <tr><td data-label="Technique">Viewport Meta Tag</td><td
data-label="Description">Adapts layout to device width.</td><td
data-label="Example">&lt;meta name="viewport" content="width=device-width,
initial-scale=1.0"&gt;</td></tr>
      <tr><td data-label="Technique">Mobile-First</td><td
data-label="Description">Design for smaller screens first.</td><td
data-label="Example">Start with base CSS, then add queries.</td></tr>
      <tr><td data-label="Technique">Breakpoints</td><td
data-label="Description">Change layout at specific widths.</td><td
data-label="Example">@media (max-width: 768px) { ... }</td></tr>
    </tbody>
  </table>
</section>

<section>
  <h2>3. CSS Media Queries</h2>
  <table>

```

```

<thead>
  <tr><th>Query</th><th>Applies When</th><th>Example</th></tr>
</thead>
<tbody>
  <tr><td data-label="Query">@media (max-width: 1200px)</td><td
data-label="Applies When">Desktop to tablet.</td><td
data-label="Example">Adjust large screens.</td></tr>
  <tr><td data-label="Query">@media (max-width: 992px)</td><td
data-label="Applies When">Tablet landscape.</td><td data-label="Example">Stack
elements.</td></tr>
  <tr><td data-label="Query">@media (max-width: 768px)</td><td
data-label="Applies When">Tablet portrait.</td><td data-label="Example">Change
layout.</td></tr>
  <tr><td data-label="Query">@media (max-width: 576px)</td><td
data-label="Applies When">Mobile phones.</td><td data-label="Example">Adjust
text & images.</td></tr>
  <tr><td data-label="Query">@media (orientation: landscape)</td><td
data-label="Applies When">Landscape mode.</td><td data-label="Example">body {
flex-direction: row; }</td></tr>
</tbody>
</table>
</section>

```

```

<section>
  <h2>4. Responsive Typography</h2>
  <table>
    <thead>
      <tr><th>Technique</th><th>Description</th><th>Example</th></tr>
    </thead>
    <tbody>
      <tr><td data-label="Technique">vw Units</td><td
data-label="Description">Scales text with viewport width.</td><td
data-label="Example">font-size: 2vw;</td></tr>
      <tr><td data-label="Technique">Clamp()</td><td
data-label="Description">Sets flexible min and max sizes.</td><td
data-label="Example">font-size: clamp(1rem, 2vw, 2rem);</td></tr>
      <tr><td data-label="Technique">Media Queries</td><td
data-label="Description">Adjusts font size by screen size.</td><td
data-label="Example">@media (max-width: 600px) { h1 { font-size: 1.5rem; }
}</td></tr>
    </tbody>
  </table>
</section>

```

```

<section>

```

```

<h2>5. Common Breakpoints</h2>
<table>
  <thead>
    <tr><th>Device</th><th>Breakpoint</th><th>Usage</th></tr>
  </thead>
  <tbody>
    <tr><td data-label="Device">Small phones</td><td
data-label="Breakpoint">max-width: 480px</td><td data-label="Usage">Mobile
layout</td></tr>
    <tr><td data-label="Device">Large phones</td><td
data-label="Breakpoint">max-width: 576px</td><td data-label="Usage">Adjust nav
& text</td></tr>
    <tr><td data-label="Device">Tablets</td><td
data-label="Breakpoint">max-width: 768px</td><td data-label="Usage">Stack grid
elements</td></tr>
    <tr><td data-label="Device">Laptops</td><td
data-label="Breakpoint">max-width: 992px</td><td data-label="Usage">Adjust
columns</td></tr>
    <tr><td data-label="Device">Desktops</td><td
data-label="Breakpoint">max-width: 1200px</td><td data-label="Usage">Standard
screen</td></tr>
  </tbody>
</table>
</section>
</main>

<footer>
  <p>Responsive CSS Cheat Sheet (c) 2025</p>
</footer>
</body>
</html>

```

cheetsheet2.css

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
}

```



```
background: #f5f5f5;
color: #333;
line-height: 1.6;
padding: 20px;
}

header {
  text-align: center;
  background: #333;
  color: #fff;
  padding: 20px;
  border-radius: 8px;
}

main {
  margin-top: 20px;
  display: grid;
  gap: 20px;
}

section {
  background: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}

h2 {
  color: #222;
  margin-bottom: 10px;
  border-bottom: 2px solid #333;
  display: inline-block;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 10px;
}

th, td {
  padding: 10px;
  border: 1px solid #ccc;
  text-align: left;
}
```

```
th {
    background: #e0e0e0;
    font-weight: bold;
}

footer {
    text-align: center;
    margin-top: 30px;
    padding: 15px;
    background: #333;
    color: #fff;
    border-radius: 8px;
}

/* ----- Mobile View: Convert Table Rows into Boxes ----- */
@media (max-width: 768px) {
    table, thead, tbody, th, td, tr {
        display: block;
        width: 100%;
    }

    thead {
        display: none;
    }

    tr {
        background: #fff;
        margin-bottom: 15px;
        border-radius: 8px;
        box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        padding: 10px;
    }

    td {
        border: none;
        padding: 8px 0;
        position: relative;
        padding-left: 110px;
        word-break: break-word;
    }

    td::before {
        content: attr(data-label);
        position: absolute;
    }
}
```

```
    left: 10px;  
    top: 8px;  
    font-weight: bold;  
    color: #333;  
    width: 95px;  
  }  
  
  body {  
    padding: 10px;  
  }  
}
```